Переменные окружения и конфигурация контейнеров

Docker Compose: .env, environment, env_file, ARG и ENV на примере REST API и PostgreSQL

Цель лекции

- Разделять код/образ приложения и настройки конкретного окружения.
- Настраивать сервисы Compose через переменные без изменения Dockerfile и app.js.
- Понимать ограничения .env и переменных окружения при хранении секретов.

Учебные вопросы

№	Вопрос
1	Конфигурация приложения и переменные окружения
2	.env и подстановка Compose
3	environment, env_file и порядок разрешения
4	ARG, ENV и конфигурация Node.js
5	Проверка, безопасность и диагностика

Откуда мы пришли: проект №8

- Nginx принимает запросы на порту 8080 и направляет их к api:3000.
- Express API использует DB_HOST=db и драйвер pg.
- PostgreSQL 17 хранит продукты в постоянном Docker Volume.
- В лекции №8 пароль и часть параметров были прописаны прямо в compose.yaml — теперь вынесем настройки.

Проблема жёстко записанных параметров

- Порт, база, логин и пароль отличаются между компьютерами и окружениями.
- Правка Dockerfile ради адреса БД приводит к ненужной пересборке.
- Нельзя считать значения, записанные в compose.yaml, секретными.
- Код и образы желательно оставлять одинаковыми при переносе.

Конфигурация и код приложения

Код и образ

- HTTP-маршруты Express
- Логика Controller и Service
- SQL-запросы
- Зависимости npm

Конфигурация среды

- Опубликованный HTTP-порт
- Адрес и порт PostgreSQL
- Название базы и пользователь
- Пароль и параметры запуска

Конфигурация приложения и переменные окружения

Чем отличается настройка от исходного кода

Схема: Код, образ и конфигурация окружения

Что неизменно в Docker, а что можно менять без изменения исходного кода

1 Исходный код

app.js

package.json

Dockerfile

nginx.conf

Файлы проекта и инструкции сборки

2 Docker-образ



- Собирается один раз
- Содержит приложение и зависимости
- Не зависит от конкретного окружения

3 Конфигурация окружения

.env

compose.yaml

environment

env_file

```
DB_HOST
DB_PORT
POSTGRES_USER
APP_PORT
```

Параметры можно менять без изменения кода и образа

4 Контейнер



Запуск образа с конкретными переменными окружения

Один образ → разные конфигурации

Тот же Docker-образ можно запускать в разных окружениях, меняя только конфигурацию.



Один и тот же Docker-образ

Разработка (dev)

Переменные окружения:

```
APP_PORT  = 3000
DB_HOST   = db-dev
```

Контейнер для разработки

Приложение доступно на порту 3000

Продакшн (prod)

Переменные окружения:

```
APP_PORT  = 8080
DB_HOST   = db-prod
```

Контейнер для продакшна

Приложение доступно на порту 8080

Что важно понять

1. Код описывает логику приложения.
2. Образ хранит приложение и зависимости.
3. Конфигурация задаёт параметры запуска.
4. Один и тот же образ можно запускать в разных окружениях.

Правильно

- Менять порт и адрес БД через переменные окружения.
- Хранить настройки отдельно от кода.

Неправильно

- Жёстко прописывать пароль БД в app.js.
- Пересобирать образ только ради смены порта.

Что такое переменная окружения

- Пара «имя = значение», переданная процессу операционной системой или средой запуска.
- Node.js читает окружение через process.env; значения в нём представлены строками.
- Переменные задаются процессу при запуске, а не автоматически синхронизируются с внешним файлом.
- Пример: DB_HOST=db, DB_PORT=5432, NODE_ENV=production.

Чтение process.env в Node.js

```javascript
const host = process.env.DB_HOST || 'db';
const port = Number(process.env.DB_PORT || 5432);
console.log({ host, port });
```

Что важно

- Строковое значение порта преобразуем через Number.
- Значения по умолчанию подходят для необязательных настроек, но не для секретов.

Почему localhost внутри контейнера не подходит

- localhost внутри api указывает на контейнер api.
- Для подключения к PostgreSQL в Compose используем DB_HOST=db.
- Имя db разрешается встроенным DNS в общей сети Docker Compose.
- Наружный порт PostgreSQL публиковать не требуется.

Примеры переменных учебного проекта

Имя	Значение в лабораторной	Для чего
HTTP_PORT	8080	Порт Nginx на хосте
DB_HOST	db	DNS-имя сервиса PostgreSQL
DB_PORT	5432	Порт PostgreSQL внутри сети
DB_NAME	shop	Название учебной базы
DB_USER	shopuser	Учебная роль PostgreSQL
DB_PASSWORD	значение задаёт студент	Пароль учебной роли

.env и подстановка значений в Compose

Файл в каталоге проекта и синтаксис ${VAR}

Что делает .env для Docker Compose

- Compose использует значения .env для подстановки в compose.yaml при построении конфигурации.
- Сам по себе .env НЕ делает все переменные окружением контейнера.
- Для передачи значения приложению применяют environment или env_file.
- Compose может также получать значения из окружения оболочки и --env-file.

Пример файла .env

```ini
HTTP_PORT=8080
DB_NAME=shop
DB_USER=shopuser
DB_PASSWORD=training_only_change_me
```

Что важно

- Это демонстрационные учебные значения; в настоящих системах пароли хранят отдельно.
- Не помещайте .env с реальными секретами в репозиторий.

Подстановка в compose.yaml

```yaml
services:
  nginx:
    ports:
      - '127.0.0.1:${HTTP_PORT:-8080}:80'
  db:
    environment:
      POSTGRES_DB: ${DB_NAME:?DB_NAME is required}
      POSTGRES_USER: ${DB_USER:?DB_USER is required}
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD is required}
```

Что важно

- ${HTTP_PORT:-8080} использует значение по умолчанию, если переменная отсутствует или пуста.
- ${DB_PASSWORD:?...} прекращает обработку конфигурации при отсутствии/пустом значении.

Схема: .env → подстановка в compose.yaml

Как Docker Compose подставляет значения из .env в конфигурацию проекта

1 Файл .env

.env

```
APP_PORT=8080
DB_HOST=db
DB_PORT=5432
POSTGRES_DB=shop
POSTGRES_USER=shopuser
POSTGRES_PASSWORD=shoppass
```

Хранит пары ИМЯ=ЗНАЧЕНИЕ для подстановки в Compose.

2 compose.yaml

YAML

```yaml
services:
  api:
    environment:
      DB_HOST: ${DB_HOST}
      DB_PORT: ${DB_PORT}

  nginx:
    ports:
      - "${APP_PORT}:80"

  db:
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

Compose читает .env и заменяет `${ПЕРЕМЕННАЯ}` на значение.

3 Результат подстановки

Итоговая конфигурация

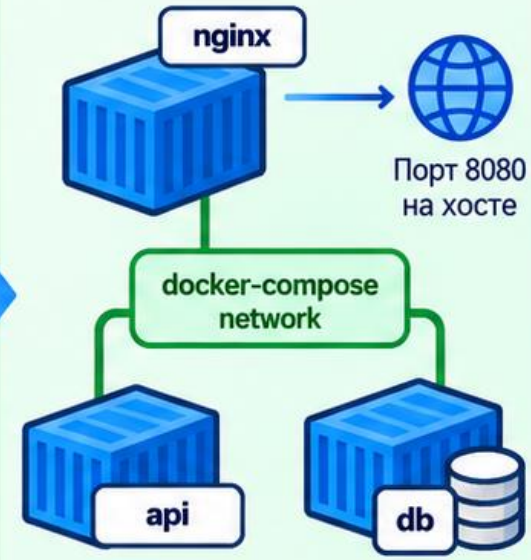
```yaml
services:
  api:
    environment:
      DB_HOST: db
      DB_PORT: 5432

  nginx:
    ports: ["8080:80"]

  db:
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shopuser
      POSTGRES_PASSWORD: shoppass
```

Перед запуском Compose формирует итоговую конфигурацию.

4 Запуск контейнеров



Сервисы получают параметры из compose.yaml.

Что важно понять

1. .env используется Compose для подстановки значений.
2. Изменение .env позволяет менять конфигурацию без правки кода.
3. Один и тот же compose.yaml можно запускать с разными значениями.
4. Переменная из .env не попадает в контейнер автоматически, если её явно не передать.

Правильно

- APP_PORT и DB_HOST хранить в .env
- Использовать `${APP_PORT}` и `${DB_HOST}` в compose.yaml
- Менять окружение без редактирования исходного кода.

Неправильно

- Жёстко прописывать значения в каждом сервисе.
- Думать, что .env сам по себе попадает внутрь контейнера.

Проверка интерполяции до запуска

BASH

```
docker compose config --environment
docker compose config --services
docker compose config
```

Что важно

- config --environment показывает переменные, используемые для интерполяции.
- config раскрывает итоговую конфигурацию и может вывести секреты; не выкладывайте полный вывод публично.

Приоритет значений при подстановке

- Если переменная задана в окружении текущей оболочки, она может переопределять .env.
- --env-file позволяет выбрать другой файл для интерполяции Compose.
- Порядок зависит от способа вызова Compose и расположения .env — всегда проверяйте результат через config.
- Не путайте подстановку Compose с фактическим окружением процесса в контейнере.

environment и env_file

Как значения действительно попадают внутрь контейнера

Схема: .env, environment и env_file — три роли

Как Docker Compose использует разные источники конфигурации

.env

Файл с переменными для подстановки в compose.yaml




```
APP_PORT=8080
DB_HOST=db
DB_PORT=5432
POSTGRES_DB=shop
POSTGRES_USER=shopuser
POSTGRES_PASSWORD=shoppass
```

- Используется Compose для подстановки значений в compose.yaml
- Сам по себе не передаётся в контейнер автоматически

environment

Явная передача переменных в сервис в compose.yaml




```
services:
  api:
    environment:
      DB_HOST: db
      DB_PORT: 5432
      NODE_ENV: production
```

- Передаёт переменные прямо в контейнер
- Удобно для явных параметров сервиса

env_file

Загрузка переменных из отдельного файла в контейнер




```
services:
  api:
    env_file:
      - api.env
```




```
NODE_ENV=development
APP_PORT=3000
LOG_LEVEL=debug
```

- Загружает набор переменных из отдельного файла
- Удобно для большого списка настроек



1. .env
Подставляет значения в compose.yaml



2. compose.yaml (environment / env_file)
Передаёт переменные в контейнер



3. Контейнер
Получает итоговое окружение

```
DB_HOST=db
DB_PORT=5432
NODE_ENV=production
...
```

Что важно понять

1. .env нужен в первую очередь Compose, а не приложению внутри контейнера.
2. environment явно задаёт переменные для контейнера.
3. env_file подгружает переменные из отдельного файла внутрь контейнера.
4. Для запуска сервиса можно сочетать .env, environment и env_file.

Правильно

- Хранить шаблон .env.example в проекте
- Передавать параметры БД через environment или env_file
- Разделять подстановку Compose и окружение контейнера

Неправильно

- Думать, что .env автоматически попадает в контейнер
- Дублировать одни и те же значения без необходимости
- Хранить реальные секреты в учебных примерах без пояснений

environment и env_file

environment

- Явно перечисляет переменные сервиса
- Допускает подстановку ${VAR}
- Удобно для нескольких ключевых параметров
- Имеет приоритет над env_file для совпадающих имён

env_file

- Читает набор переменных из отдельного файла
- Путь указывается относительно compose.yaml
- Удобно для группы обычных настроек
- Не предназначен сам по себе для защиты секретов

Файл api.env

```ini
DB_HOST=db
DB_PORT=5432
APP_LABEL=course9-api
```

Что важно

- Этот файл содержит несекретные настройки именно приложения API.
- Compose передаёт эти значения в окружение контейнера api.

Передача настроек контейнеру api

```yaml
services:
  api:
    build: ./api
    env_file:
      - ./api.env
    environment:
      DB_NAME: ${DB_NAME:?required}
      DB_USER: ${DB_USER:?required}
      DB_PASSWORD: ${DB_PASSWORD:?required}
```

Что важно

- env_file читает DB_HOST, DB_PORT, APP_LABEL.
- environment получает параметры через интерполяцию и передаёт контейнеру.

Проверка настроек без печати пароля

```bash
docker compose exec api node -e \
"console.log({ \
  host: process.env.DB_HOST, \
  port: process.env.DB_PORT, \
  db: process.env.DB_NAME, \
  passwordSet: Boolean(process.env.DB_PASSWORD) \
})"
```

Что важно

- passwordSet показывает только наличие непустого значения.
- Не выводите реальные пароли в терминальные журналы и скриншоты.

Что не делает env_file

- Не создаёт автоматически отдельный том или сетевой сервис.
- Не меняет настройки уже запущенного процесса до пересоздания соответствующего контейнера.
- Не шифрует переменные: окружение и метаданные контейнера доступны администраторам Docker.
- Не подставляет DB_HOST в браузерный JavaScript — это серверное окружение.

ARG и ENV в Dockerfile

Настройки сборки и времени выполнения

ARG, ENV, environment: когда используются

Механизм	Этап	Особенность
ARG	Сборка образа	Параметр сборки; не подходит для секретов
ENV	Образ и контейнер	Значение сохраняется в конфигурации образа
environment	Запуск сервиса Compose	Передаёт/переопределяет переменные в контейнере
env_file	Запуск сервиса Compose	Импортирует несколько переменных из файла

Dockerfile: ARG и ENV

DOCKERFILE

```
FROM node:22-alpine
ARG APP_BUILD_LABEL=course9
ENV NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Что важно

- ARG доступен при сборке; APP_BUILD_LABEL здесь служит только демонстрацией синтаксиса.
- ENV NODE_ENV задаёт значение по умолчанию для процессов контейнера.
- Не передавайте реальные пароли через ARG или ENV.

Что важнее для нашего проекта

- Адрес DB_HOST и порт DB_PORT нужны при выполнении, а не при сборке образа.
- Настройки БД передаются через Compose: env_file и environment.
- NODE_ENV=production можно оставить в Dockerfile как несекретную характеристику среды.
- Изменение опубликованного порта Nginx не требует пересборки Express-образа.

Проверка, безопасность и диагностика

Проверяем фактическое окружение, не раскрывая секреты

Схема: Проверка конфигурации и поиск ошибок

Как проверить переменные окружения, итоговую конфигурацию и найти типичные ошибки в Docker Compose

1 .env и compose.yaml

Исходные параметры проекта

.env

```
APP_PORT=8080
DB_HOST=db
DB_PORT=5432
POSTGRES_DB=shop
```

compose.yaml

```
services:
  api:
    image: myapp:latest
    env_file: .env
    environment:
      - DB_HOST=${DB_HOST}
```

2 Проверка итоговой конфигурации

```
$ docker compose config
```

Показывает итоговую конфигурацию после подстановки

```
$ docker compose config -q
```

Проверяет корректность без запуска

В итоговой конфигурации вы увидите уже подставленные значения из .env (например, порты, имена, переменные и т. д.).

3 Проверка контейнеров и переменных




```
docker compose ps
```

Состояние сервисов

```
docker compose logs api
```

Логи приложения

```
docker compose exec api env
```

Переменные внутри контейнера

```
docker compose exec api
printenv DB_HOST
```

Проверка конкретной переменной

4 Проверка подключения и поиск ошибок

```
$ curl http://localhost:8080/api/products
```

Проверка ответа приложения

```
$ docker compose exec api ping db
```

Проверка сетевого соединения с БД

```
$ docker compose exec api nc -zv db 5432
```

Проверка доступности порта БД

Типичные проблемы

- Неверный DB_HOST (например, localhost вместо db)
- Неверный APP_PORT (приложение недоступно)
- Контейнер не запущен (service exited или не стартует)
- Ошибка в синтаксисе YAML (неправильные отступы)
- Переменная из .env не передана в контейнер (нет в environment или env_file)

Что важно понять

- .env нужен Compose для подстановки значений
- environment и env_file передают переменные в контейнер
- Сначала проверяют config, затем ps и logs
- Типичная ошибка: думать, что .env сам попадает в контейнер

Правильно

- Проверять docker compose config перед запуском
- Смотреть логи нужного сервиса
- Сверять переменные внутри контейнера

Неправильно

- Искать ошибку только в коде
- Путать .env и env_file
- Менять пароль БД и ждать, что старый том обновится сам

Как обновлять настройки проекта

1	**Шаг 1**	1. Изменить переменную в .env или api.env.
2	**Шаг 2**	2. Проверить синтаксис: docker compose config -q.
3	**Шаг 3**	3. Пересоздать затронутый сервис: docker compose up -d --force-recreate api.
4	**Шаг 4**	4. Проверить статус и логи контейнера.
5	**Шаг 5**	5. Проверить GET /api/products через Nginx.

Набор диагностических команд

BASH

```
docker compose config -q
docker compose ps
docker compose logs --tail=30 api
docker compose logs --tail=30 db
curl -i http://127.0.0.1:8080/api/products
```

Что важно

- -q проверяет конфигурацию без вывода полного значения пароля.
- Номер порта в curl должен совпадать с HTTP_PORT из .env.

Почему пароль в .env не равен безопасному секрету

- Файл .env — обычный текстовый файл и может попасть в резервную копию или Git.
- Переменные окружения можно увидеть через Docker API / docker inspect при соответствующих правах.
- Для production используют специализированные хранилища секретов или механизмы секретов Compose, если приложение их поддерживает.
- Docker-группа предоставляет привилегированный доступ — не раздавайте его без необходимости.

Пример .gitignore

```text
.env
.env.*
!.env.example
node_modules/
```

Что важно

- Шаблон .env.example содержит названия переменных без настоящих паролей.
- Правила .gitignore не удаляют уже опубликованные секреты из истории Git.

Проблема: api не подключается к PostgreSQL

- Проверьте имя DB_HOST: в Compose оно должно быть db.
- Проверьте DB_NAME/DB_USER/DB_PASSWORD и фактические настройки существующего тома.
- Изменение POSTGRES_PASSWORD в .env НЕ меняет пароль уже созданной роли в существующей БД.
- Если меняете пароль для существующей БД, нужен SQL ALTER ROLE или новый изолированный том.

Диагностический эксперимент: неверный DB_HOST

1	**Шаг 1**	1. Изменить DB_HOST в api.env на db-wrong.
2	**Шаг 2**	2. Пересоздать только api и выполнить GET /api/products.
3	**Шаг 3**	3. Сравнить сообщение API/логи api; ошибка должна указывать на соединение или DNS.
4	**Шаг 4**	4. Вернуть DB_HOST=db.
5	**Шаг 5**	5. Пересоздать api и подтвердить успешный GET.

Что сохраняется при пересоздании

- Настройки контейнера обновляются при пересоздании сервиса.
- Docker Volume с данными PostgreSQL остаётся на месте при обычном down/up.
- Команда docker compose down -v может удалить тома вместе с учебными данными.
- Изменение .env не требует автоматически пересоздавать PostgreSQL при каждой правке, если его конфигурация не менялась.

Ограничения учебного решения

- В лабораторной используем демонстрационный пароль и локальное опубликование Nginx на 127.0.0.1.
- Передача секретов через environment применяется только ради изучения Compose.
- Для развертывания вне учебного стенда потребуются секреты, права, TLS и управление доступом.
- Не переносите учебный пароль или compose.yaml в production без ревизии.

Итоги лекции

1. Переменные окружения отделяют конфигурацию от кода, но не заменяют управление секретами.
2. .env обеспечивает интерполяцию Compose; environment/env_file передают значения сервису.
3. ARG относится к сборке, ENV задаёт переменные образа/контейнера.
4. docker compose config -q проверяет конфигурацию; реальные значения можно осмотреть безопасно.
5. При смене настроек пересоздают контейнер; отдельный том БД сохраняет данные.

Контрольные вопросы

1. Чем переменные окружения отличаются от констант в исходном коде?
2. Делает ли наличие .env переменные доступными внутри контейнера автоматически?
3. Чем environment отличается от env_file?
4. Что означает ${HTTP_PORT:-8080}?
5. Что делает ${DB_PASSWORD:?required}?
6. Почему DB_HOST=db, а не localhost?
7. Когда применяются ARG и ENV?
8. Почему нельзя хранить пароли в ARG?
9. Как проверить YAML без печати секретов?
10. Почему изменение POSTGRES_PASSWORD не меняет пароль в уже инициализированном томе?